

Worksheet — 2.3 Algorithms

Name: ___ Date: ____

OCR A Level Computer Science H446, Component 02, Section 2.3.1 — Algorithms.

Time: ~60 min. SHOW YOUR TRACES.

Section A — Skills practice

Activity 1 — Big O match

Match each algorithm to its **best**, **average** and **worst** time complexity by writing the correct Big O in each cell. Choose from: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$.

Algorithm	Best	Average	Worst
Linear search			
Binary search			
Bubble sort (with swap flag)			
Insertion sort			
Merge sort			
Quick sort			

Activity 2 — Trace a sort (bubble OR insertion)

Sort this list into **ascending** order: [5, 1, 4, 2, 8]

Circle which you are tracing: **BUBBLE / INSERTION**

Fill in the list state **after each pass**. Add rows if you need them.

Pass	List state after this pass	Swaps this pass
Start	5 1 4 2 8	—
1	— — — — —	
2	— — — — —	
3	— — — — —	
4	— — — — —	

Final sorted list: _ _ _ _ _

Activity 3 — Trace a binary search

Sorted list (index 0–8):

Index	0	1	2	3	4	5	6	7	8
Value	3	7	11	15	19	23	27	31	35

Search for 27. Use `mid = (low + high) DIV 2` (integer division, round down). Fill the table.

Step	low	high	mid (index)	value at mid	< = > target?
1					
2					
3					
4					

Found at index: Number of comparisons:

Activity 4 — Big O justification (short answer)

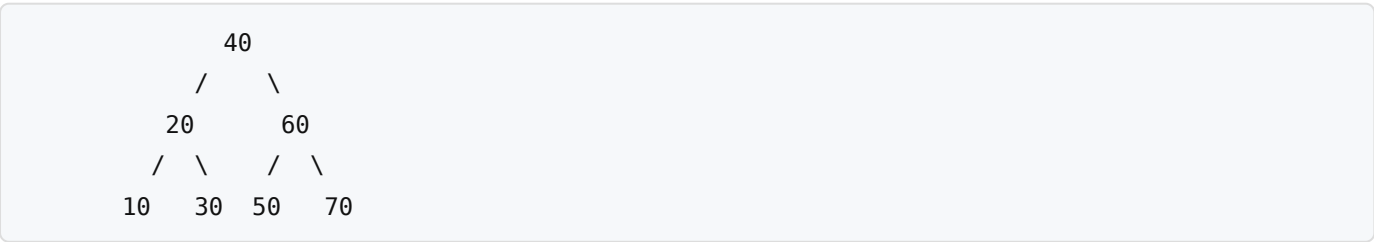
(a) Explain why binary search is **$O(\log n)$** . Refer to what happens to the list each comparison.

(b) Explain why bubble sort is **$O(n^2)$** in the average case.

(c) State the precondition binary search requires that linear search does not.

Activity 5 — Tree traversals

Binary tree (text form):



Write the output of each traversal:

Pre-order (Node → Left → Right):

In-order (Left → Node → Right):

Post-order (Left → Right → Node):

Which traversal outputs a BST in ascending order? ____

Activity 6 — Dijkstra's shortest path

Weighted, undirected graph given as an **edge list** (node1 — node2 : weight):

A – B : 4
A – C : 2
B – C : 1
B – D : 5
C – D : 8
C – E : 10
D – E : 2
D – F : 6
E – F : 3

Find the shortest path and total distance from A to F.

Working table — write the tentative distance to each node, updating as you settle nodes. Cross out / overwrite when a shorter route is found.

Settled node (order)	A	B	C	D	E	F
Initial	0	∞	∞	∞	∞	∞

Shortest path A → F: _____ → _____ → _____ → _____ → _____

Total distance: _____

Challenge

Challenge box. A* search uses $f(n) = g(n) + h(n)$.

(a) For the graph above, suppose you are given a straight-line heuristic estimate to F of: $h(A)=9$, $h(B)=7$, $h(C)=8$, $h(D)=4$, $h(E)=3$, $h(F)=0$. Is this heuristic **admissible**? Explain how you would check, and what admissible means.

(b) State ONE situation where A explores fewer nodes than Dijkstra, and ONE condition under which A behaves exactly like Dijkstra.

...

.....

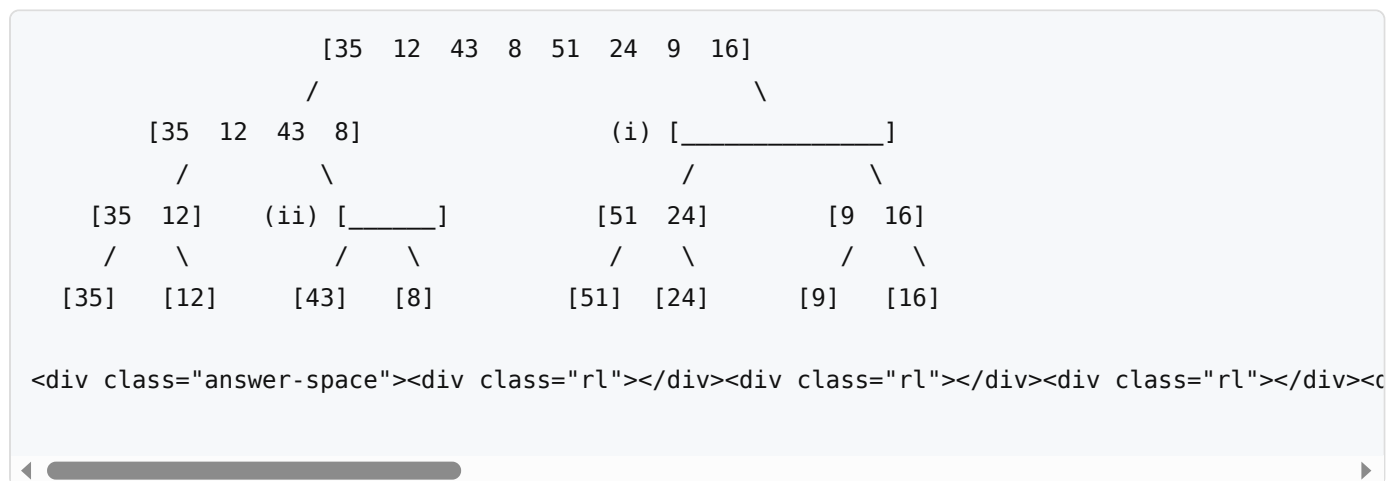
...

Activity 7 — Merge sort: complete the divide and merge diagram

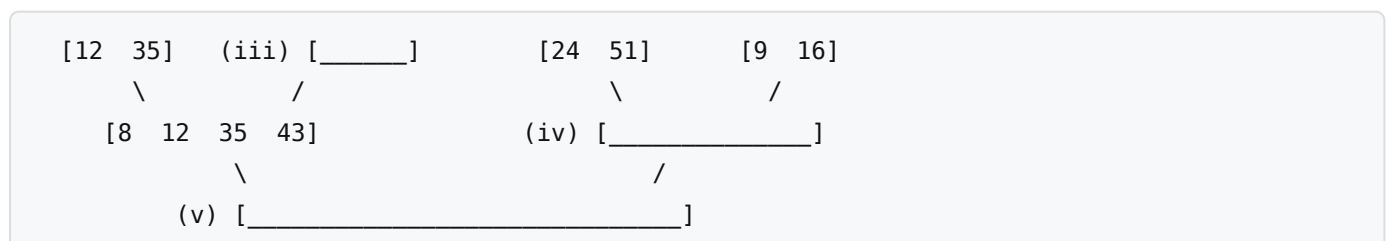
Merge sort the list [35, 12, 43, 8, 51, 24, 9, 16].

(a) Complete the missing boxes (i)–(v) in the diagram.

DIVIDE (halve repeatedly until every sublist has one element):



MERGE (combine pairs back together, in order, comparing front elements):



(b) For this list of 8 items, how many **levels of dividing** are needed to reach single-element lists? ___ How is this number related to $n = 8$? _____

(c) Approximately how much work (comparisons/copies) is done at **each merge level**, in terms of n ? ____
Hence state why merge sort is $O(n \log n)$.

(d) Unlike quicksort, merge sort's worst case is still $O(n \log n)$. Give the **cost** merge sort pays for this reliability. ____

Activity 8 — A* vs Dijkstra: which and why?

Weighted, undirected graph (edge list) and a heuristic estimate $h(n)$ of the distance from each node to the goal **G**:

S – A : 1 S – B : 4 S – D : 2
A – B : 2 A – C : 5
B – C : 2 C – G : 3

Node	S	A	B	C	D	G
$h(n)$	7	6	4	3	9	0

(a) **Dijkstra from S**. Complete the table (settle the unvisited node with the smallest distance each round) and write the **order the nodes are settled**.

Settled (order)	S	A	B	C	D	G
Initial	0	∞	∞	∞	∞	∞

Settle order: _ → _ → _ → _ → _

(b) **A* from S to G**. Each time you expand a node, record it with its g , h and $f = g + h$. Always expand the open node with the **lowest f** . Stop when G is expanded.

Node expanded	$g(n)$	$h(n)$	$f(n)$
S	0	7	7

Shortest path S → G: ____ → ____ → ____ → ____ Distance: ____

(c) Which node does **Dijkstra settle** but **A* never expands**? ____ Use its f value to explain why A* ignores it.

(d) Decision. A satnav needs the route from one start to **one destination** on a map where straight-line distances are known. A network-monitoring tool needs the shortest routes from one server to **every** other node. State, with a reason, which algorithm suits each task.

Section B — Exam practice (30 marks)

Answer all questions. Questions marked with an asterisk () assess the quality of your extended response. All code must be written in OCR Exam Reference Language.*

Q1. State the **worst-case** time complexity of quicksort in Big O notation, and describe a situation in which this worst case occurs. **[2]** (AO1)

Q2. The following procedure is written in OCR Exam Reference Language, where `n` is the number of elements in the array `arr`.

```
procedure pairs(arr:array, n)
  for i = 0 to n - 1
    for j = 0 to n - 1
      print(arr[i] + arr[j])
    next j
  next i
endprocedure
```

Determine, with justification, the **time complexity** of this procedure in Big O notation. **[3]** (AO2)

Q3. The following algorithm searches the sorted array `arr = [4, 9, 15, 22, 30, 41, 56, 70]` (indexes 0–7) for `target = 10`.

```

low = 0
high = 7
found = false
while low <= high AND found == false
    mid = (low + high) DIV 2
    if arr[mid] == target then
        found = true
    elseif arr[mid] < target then
        low = mid + 1
    else
        high = mid - 1
    endif
endwhile
if found then
    print(mid)
else
    print(-1)
endif

```

(i) Complete the trace table, adding one row for each iteration of the while loop. **[3]** (AO2)

low	high	mid	arr[mid]	found
0	7			

(ii) State the value printed when the algorithm ends. **[1]** (AO2)

Q4. A sorted array `arr` contains `n` integers (zero-indexed). Write a function `searchIndex(arr, n, target)` using **OCR Exam Reference Language** that performs a **binary search** and returns the index of `target` in `arr`, or `-1` if it is not present. **[5]** (AO3)

Q5. State the data structure used to implement:

(i) a **breadth-first** traversal of a graph. **[1]** (AO1)

(ii) a **depth-first** traversal of a graph. **[1]** (AO1)

Q6. A graph is stored as the following adjacency list:

```
A : B, C
B : A, D
C : A, E
D : B, E
E : C, D
```

(i) A breadth-first traversal starts at A and visits neighbours in alphabetical order. State the order in which the nodes are visited. **[2]** (AO2)

(ii) The edges of this graph are unweighted. Explain why a breadth-first traversal from A finds a path to E using the **fewest edges**, while a depth-first traversal may not. **[1]** (AO2)

Q7. The A *pathfinding* algorithm assigns each node a value $f(n) = g(n) + h(n)$. State what $g(n)$ and $h(n)$ each represent. **[2]*** (AO1)

Q8* A music streaming service stores 50 million track records, held in order of track ID. Users search for tracks by ID thousands of times per second. Each night a batch of a few thousand new tracks is added and the full file must be put back in order; the nightly process runs on a server with a large amount of memory. A cut-down version of the software must also run on an embedded car system with very limited memory, where it keeps a few hundred downloaded tracks in order.

Discuss which **searching and sorting algorithms** the developers should choose for this system, justifying your choices with reference to **time and space complexity**. [9] (AO3)

This image shows a blank sheet of white paper with horizontal blue ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.